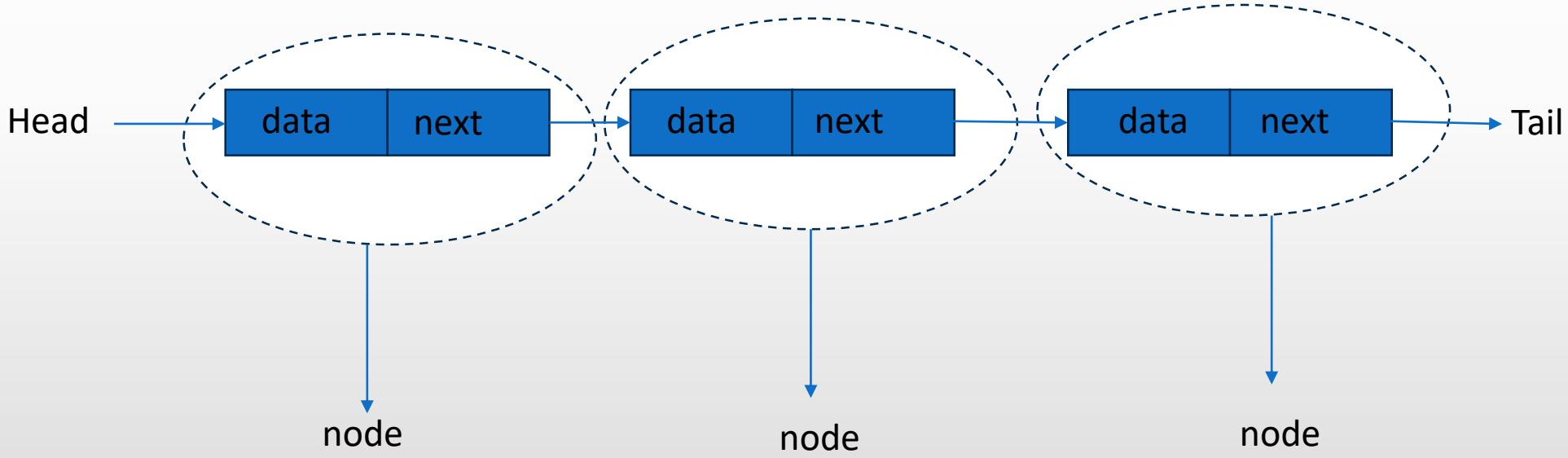
Data Structure & Algorithms

Linked List

Introduction

A linked list is a linear data structure.

It consists of nodes where each node contains data and a reference (link) to the next node in the sequence.
This allows for dynamic memory allocation and efficient insertion and deletion operations compared to arrays.



Node Structure: A node in a linked list typically consists of two components:

Data: It holds the actual value or data associated with the node.

Next Pointer: It stores the memory address (reference) of the next node in the sequence.

Head and Tail: The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or nullptr, indicating the end of the list. This node is known as the tail node.

Uses of Linked List

- The list is not required to be contiguously present in the memory. The node can reside any where in the memory and linked together to make a list. This achieves optimized utilization of space.
- list size is limited to the memory size and doesn't need to be declared in advance.
- Empty node can not be present in the linked list.
- We can store values of primitive types or objects in the singly linked list.

Why use linked list over array?

Array contains following limitations:

- The size of array must be known in advance before using it in the program.
- Increasing size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.
- All the elements in the array need to be contiguously stored in the memory. Inserting any element in the array needs shifting of all its predecessors.

Linked list is the data structure which can overcome all the limitations of an array. Using linked list is useful because,

- It allocates the memory dynamically. All the nodes of linked list are non-contiguously stored in the memory and linked together with the help of pointers.
- Sizing is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

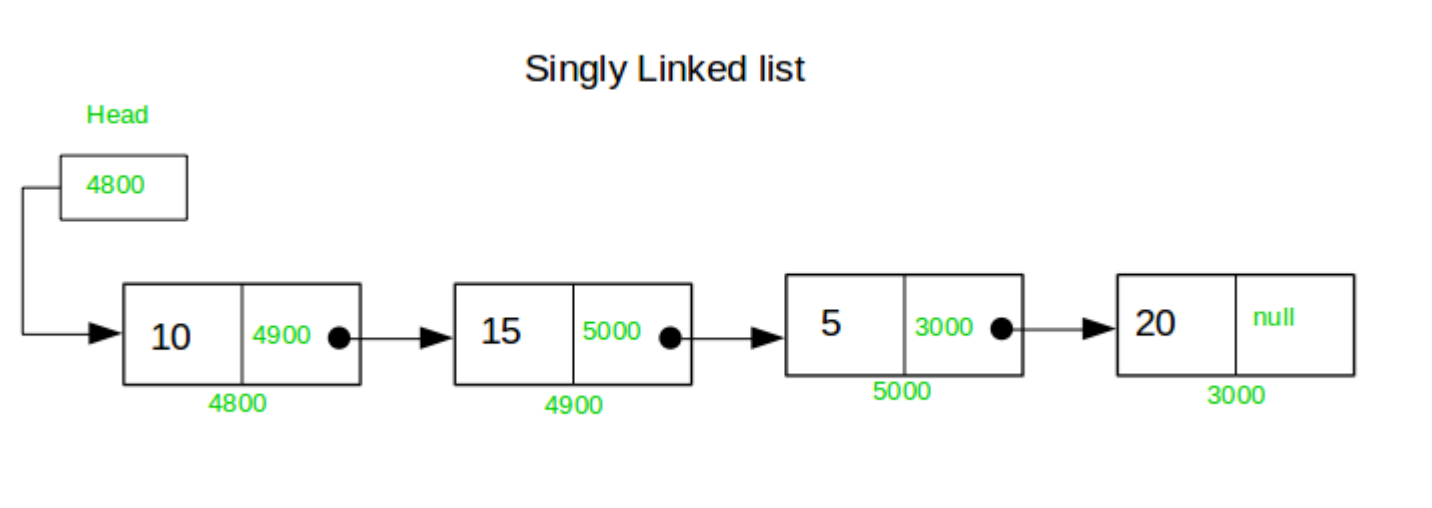
Types of Linked List

There are three common types of Linked List.

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Singly Linked List

It is the most common. Each node has data and a pointer to the next node.



One way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.

Representation of Linked List

how each node of the linked list is represented. Each node consists:

- A data item
- An address of another node

We wrap both the data item and the next node reference in a struct as:

```
struct node
{
   int data;
   struct node *next;
};
```

Operations on Linked List

Node Creation

```
struct node
{
    int data;
    struct node *next;
};
struct node *head, *ptr;
ptr = (struct node *)malloc(sizeof(struct node *));
```

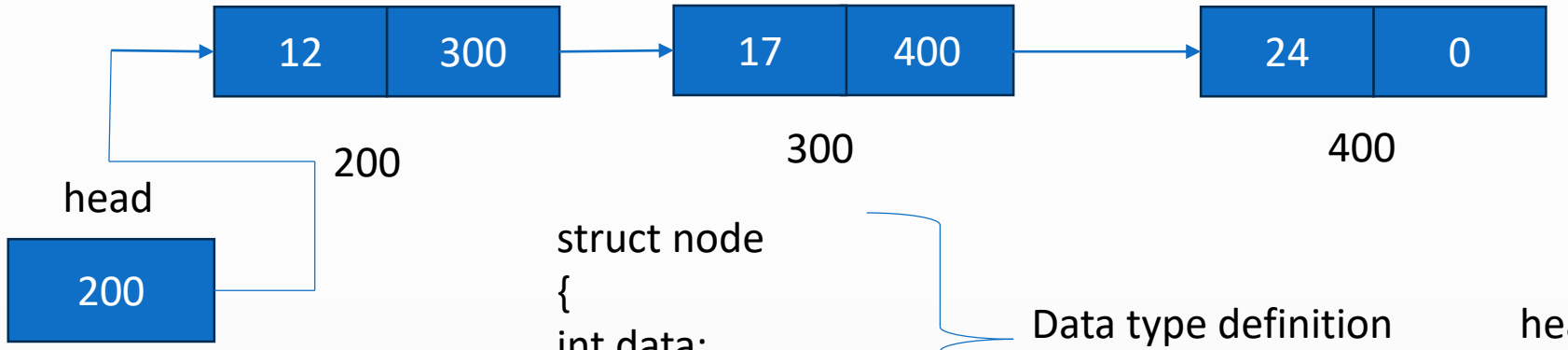
Traversal in Linked List

```
traverse(struct node* head)
{
    if(head==null)
    printf(“linked list empty”);
    struct node *ptr;
        ptr=head;
    while(ptr!=null)
        {

            ptr=ptr->next;

        }
}
```

Linked List Implementation in C

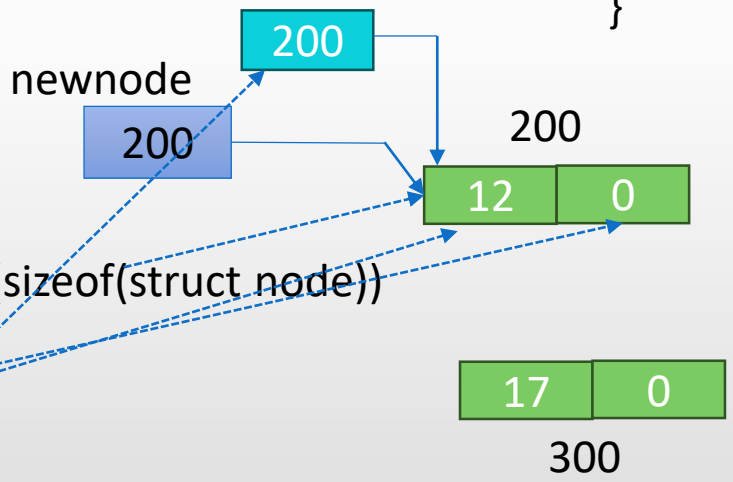



```
struct node
{
int data;
struct node *next;
};
```

Data type definition

```
struct node *head, *newnode,*temp;
head=0;
        newnode =(structnode*)malloc(sizeof(struct node))
printf("enter data");
 scanf("%d", &newnode⟶data);
newnode ⟶next=0;
if(head==0)
{
head=temp=newnode;}
```

```
else
{
head⟶next=newnode;
}
```



Insertion in Linked List

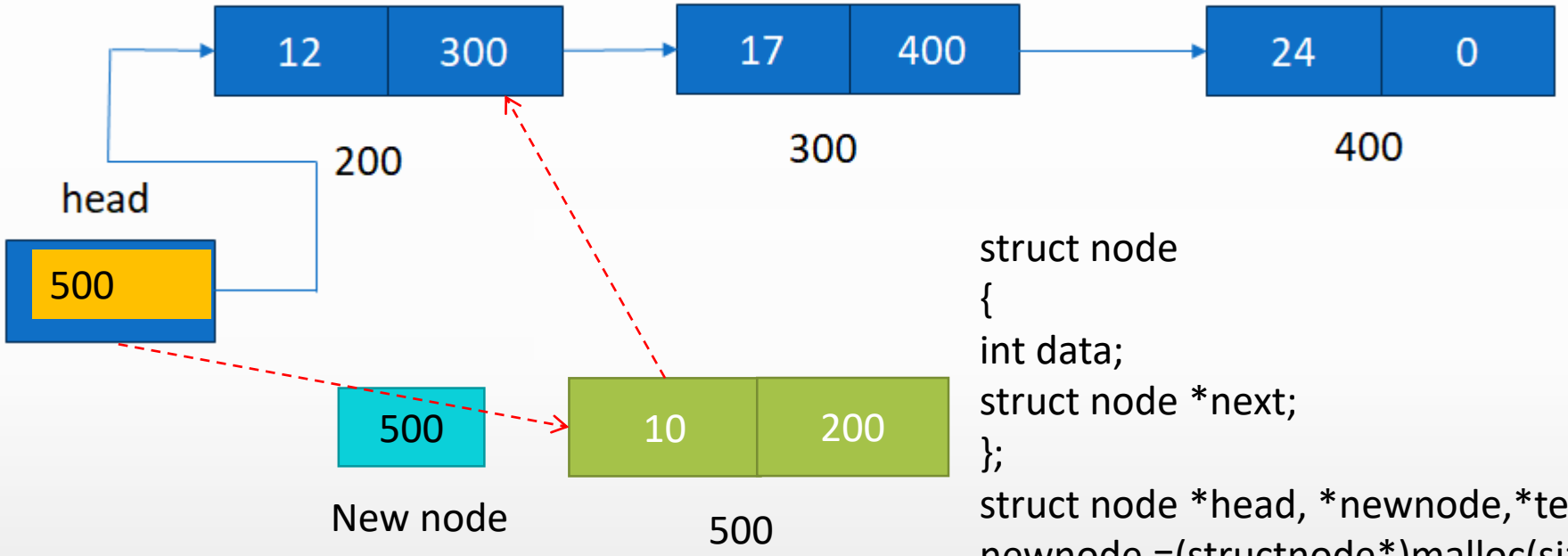
The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

1.	Insertion at beginning	It involves inserting any element at the front of the list.
2.	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one.
3.	Insertion after specified node	It involves insertion after the specified node of the linked list.

Insertion at beginning in Linked List

```
insertbeg(struct node *head, int info)
   {
   struct node *new;
   new=(struct node*) malloc(size of (struct node));
   new ->data=info;
   new ->next =head;
   head=new;
   return head;
   }
```

Insertion at the beginning




```
struct node
{
int data;
struct node *next;
};
struct node *head, *newnode,*temp;
newnode =(structnode*)malloc(sizeof(struct node))
printf("enter data");
 scanf("%d", &newnode ->data);
```

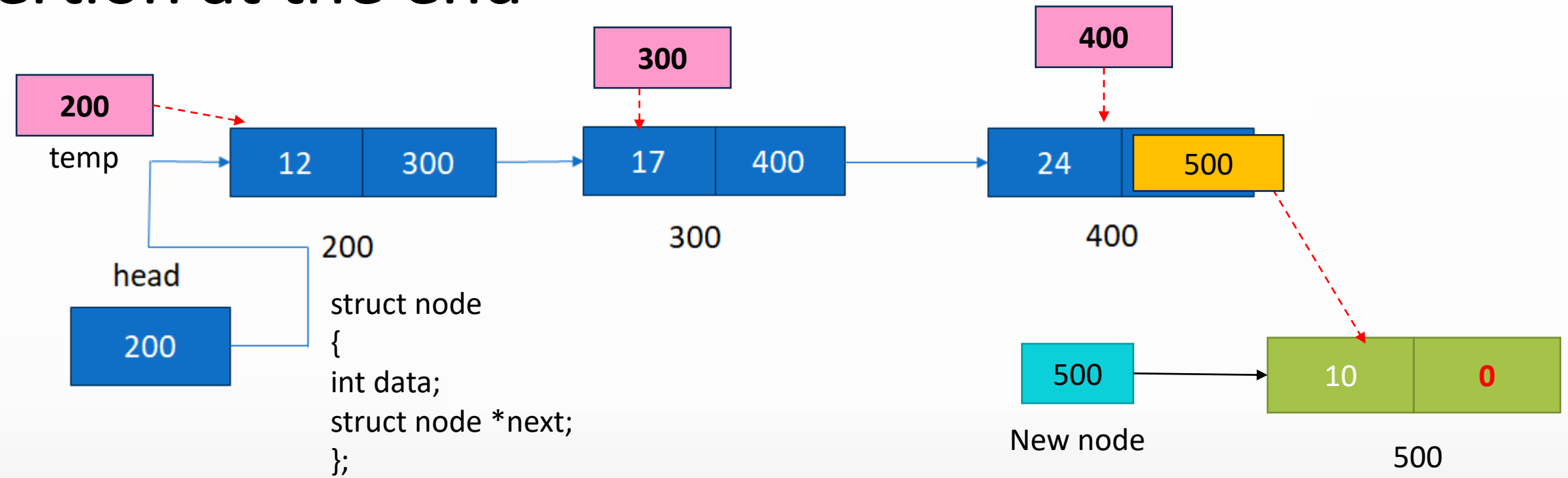
```
newnode->next=head;
```

```
head=newnode;
```

Insertion at end in Linked List

```
insert_end(struct node*head, int info)
{
struct node *ptr *new;
new=(struct node*) malloc(size of (struct node));
new ->data=info;
new ->next = null;
ptr=head;
if(ptr!=null)
{
while(ptr->next != null)
{ptr = ptr->next;
}
ptr->next = new;
}
else
head=new;
return head;
}
```

Insertion at the end




```
struct node
{
int data;
struct node *next;
};
struct node *head, *newnode,*temp;
newnode =(structnode*)malloc(sizeof(struct node))
printf("enter data");
 scanf("%d", &newnode ->data);
```

newnode->next=null;
temp=head;

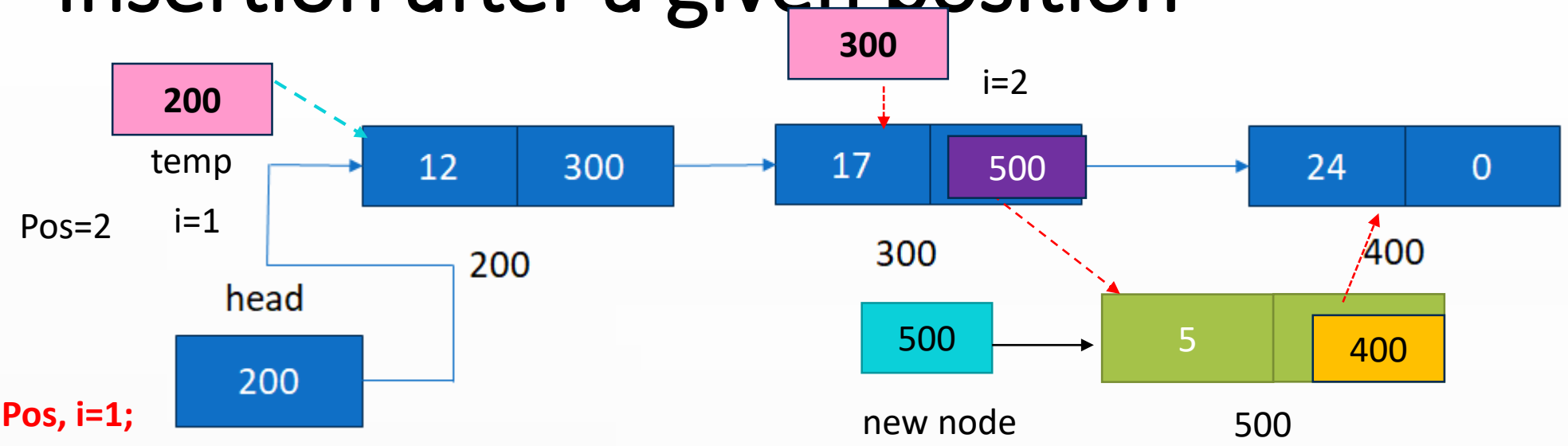
```
while temp->next!=0
  {
  temp=temp->next;
  }
temp->next=newnode;
```

Insertion after give position in Linked List

```
insert_after(struct node*head, int x, int info)
{
struct node *ptr *new;
new=(struct node**) malloc(size of (struct node));
new ->data=info;
ptr = head;
while(ptr->data != x && ptr != null)
        {
           ptr= ptr->next

        }
if(ptr->data == x)
     {
        new->next = ptr->next;
        ptr->next = new;
      }
   return head;
   }
```

Insertion after a given position




```
int Pos, i=1;
struct node
{
int data;
struct node *next;
};
struct node *head, *newnode,*temp;
newnode =(structnode*)malloc(sizeof(struct node));
printf("enter data");
 scanf("%d", &newnode ->data);
printf("enter position");
 scanf("%d", &Pos);
```

```
if Pos>count
{
printf("invalid Position");
}
else
{temp=head;
    while(i<Pos)
    {
    temp=temp->next;
    i++;}
```

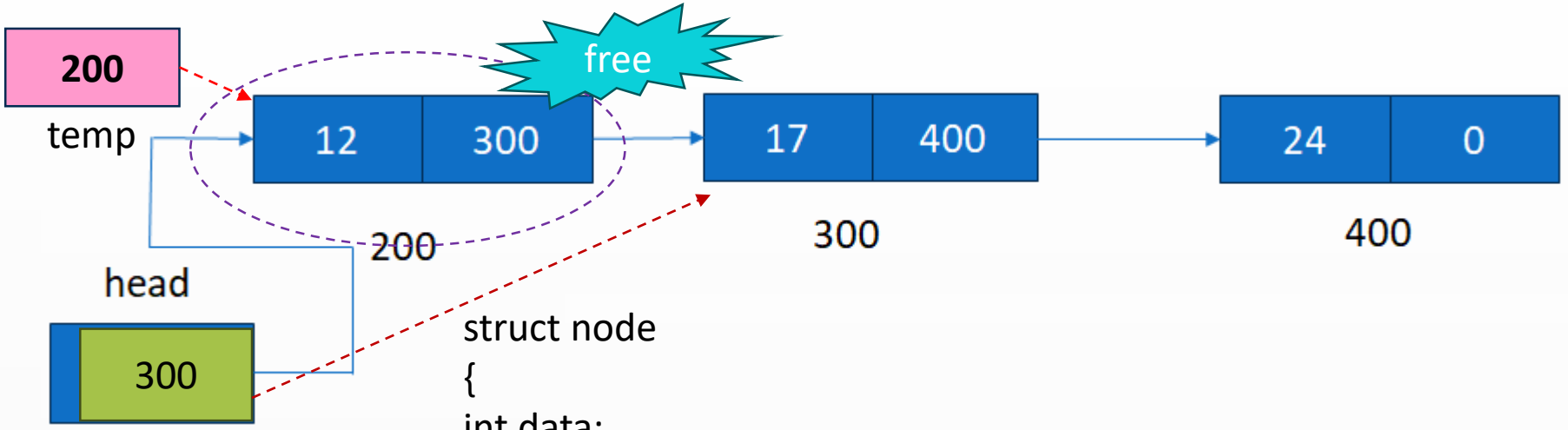
```
newnode->next=temp->next;
temp->next=newnode;
```

Deletion in Linked List

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

1.	Deletion at beginning	Removing the node from beginning of the list
2.	Deletion at the end	Removing the node from end of the list.
3.	Deletion after specified node	It involves deleting the node after the specified node in the list.

Deletion from beginning



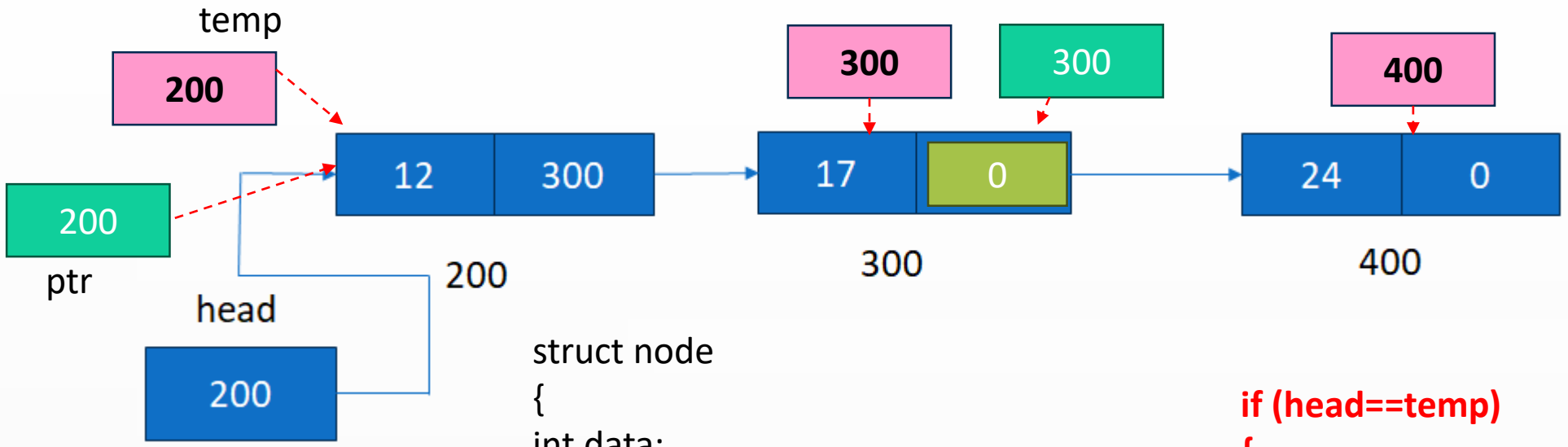

```
struct node
{
int data;
struct node *next;
};
struct node *head, *temp;
```

temp=head;

head=head->next;

free(temp);

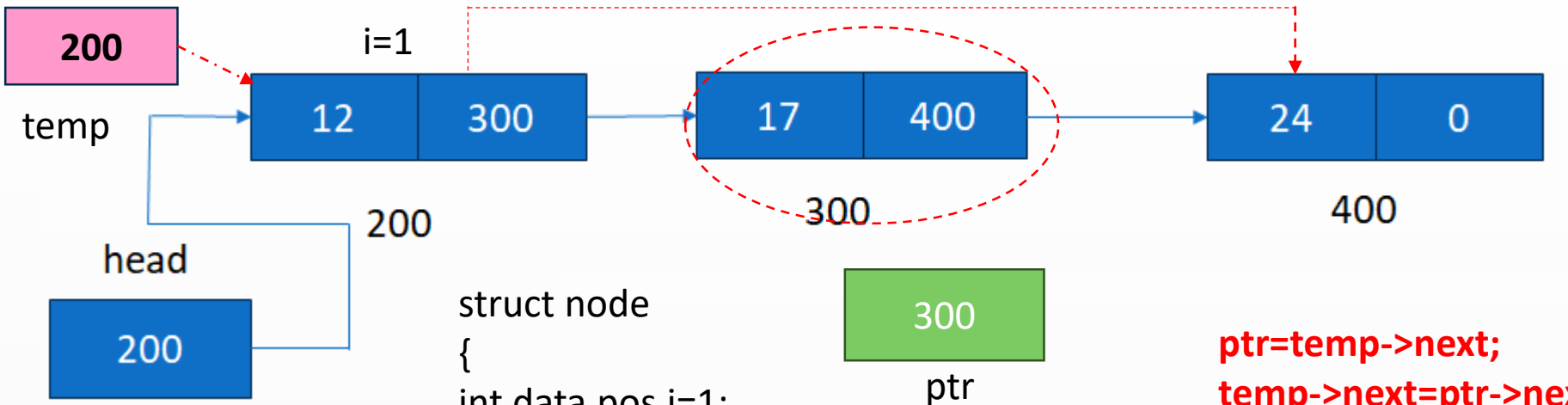
Deletion from end




```
struct node
{
int data;
struct node *next;
};
struct node *head, *temp, *ptr;
temp=head;
while( temp->next!=0)
{
ptr=temp;
temp=temp->next;
}
```

```
if (head==temp)
{
head=0;}
else
{
ptr->next=0;
}
free(temp);
```

Deletion from specified Position




```
struct node
{
int data,pos,i=1;
struct node *next;
};
struct node *head, *temp, *ptr;
```

```
temp=head;
printf("Enter Position");
scanf("%d", &pos          //pos=2

while(i<pos)
{
temp=temp->next;
i++;
}
```

```
ptr=temp->next;
temp->next=ptr->next;
free(ptr);
```

Thank You